

30 | Unsafe编程（下）：使用Rust为Python写一个扩展

唐刚 · Rust 语言从入门到实战



你好，我是 Mike。

上一讲我们了解了 Unsafe Rust 的所属定位和基本性质，这一讲我们就来看看 Rust FFI 编程到底是怎样一种形式。

FFI 是与外部语言进行交互的接口，在 Safe Rust 看来，它是不可信的，也就是说 Rust 编译不能保证它是安全的，只能由程序员自己来保证，因此在 Rust 里调用这些外部的代码功能就需要把它们包在 `unsafe {}` 中。

Rust 和 C 有血缘关系，具有 ABI 上的一致性，所以 **Rust 和 C 之间可以实现双向互调，并且不会损失性能。**

Rust 调用 C

我们先来看在 Rust 中如何调用 C 库的代码。

一般各个平台下都有 libm 库，它是操作系统基本的数学 math 库。下面我们以 Linux 为例来说明。下面的示例代码来自 [🔗 Rust By Example](#)。

[📄 复制代码](#)

```
1 use std::fmt;
2
3 // 连接到系统的 libm 库
4 #[link(name = "m")]
5 extern {
6     // 这是一个外部函数，计算单精度复数的方根
7     fn csqrtf(z: Complex) -> Complex;
8     // 计算复数的余弦值
9     fn ccosf(z: Complex) -> Complex;
10 }
11
12 // 对unsafe调用的safe封装，从此以后，就按safe函数方式使用这个接口
13 fn cos(z: Complex) -> Complex {
14     unsafe { ccosf(z) }
15 }
16
17 fn main() {
18     // z = -1 + 0i
19     let z = Complex { re: -1., im: 0. };
20
21     // 调用m库中的函数，需要用 unsafe {} 包起来
22     let z_sqrt = unsafe { csqrtf(z) };
23
24     println!("the square root of {:?} is {:?}", z, z_sqrt);
25
26     // 调用安全封装后的函数
27     println!("cos({:?}) = {:?}", z, cos(z));
28 }
29
30 // 用 repr(C) 标注，定义Rust结构体的ABI格式，按C的ABI来
31 #[repr(C)]
32 #[derive(Clone, Copy)]
33 struct Complex {
34     re: f32,
35     im: f32,
36 }
37
38 // 实现复数的打印输出
39 impl fmt::Debug for Complex {
40     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
41         if self.im < 0. {
42             write!(f, "{}-{}i", self.re, -self.im)
```

```

43         } else {
44             write!(f, "{}+{}i", self.re, self.im)
45         }
46     }
47 }

```

libm 库是用 C 语言实现的，我们要调用它，需要用这样的标注。

[复制代码](#)

```

1  #[link(name = "m")]
2  extern {
3  }

```

用 `link` 属性宏将 libm 库连接进来，以便去里面找对应的函数。然后把需要的外部函数导入进来。你可以看一下 `csqrtf()` 函数和 `ccosf()` 的 C 语言签名。

[复制代码](#)

```

1  float complex csqrtf(float complex z);
2  float complex ccosf (float complex z);

```

导入的时候，要翻译成 Rust 函数的签名形式。

[复制代码](#)

```

1  fn csqrtf(z: Complex) -> Complex;
2  fn ccosf(z: Complex) -> Complex;

```

调用这两个函数的时候，都需要用 `unsafe {}` 包起来。

然后我们仔细看一下在 Rust 中对应到 libm 中的复数的定义。

[复制代码](#)

```

1  #[repr(C)]
2  #[derive(Clone, Copy)]
3  struct Complex {

```

```
4     re: f32,  
5     im: f32,  
6 }
```

对应的 C 语言中的结构定义在 [complex.h](#) 头文件中，它是 C99 标准引入的数据类型。

本身这个定义非常简单，就是定义复数的实部和虚部就可以了，不过一定要用 `#[repr(C)]` 标注。它的意思是这个类型编译的时候，要使用 C 语言的 ABI 格式。我们一般都使用 C 语言 ABI 格式，但是也有一些其他的 ABI 格式，你可以点击我给出的 [链接](#)，查到 Rust 中支持的 ABI 格式列表。

一般来讲，我们会使用 Rust 函数对这些外部 unsafe 函数做一下封装。像 cos 函数这样，你可以参看一下示例代码。

```
1 fn cos(z: Complex) -> Complex {  
2     unsafe { ccosf(z) }  
3 }
```

[复制代码](#)

执行 `cargo run`，输出了下面这两行代码。

```
1 the square root of -1+0i is 0+1i  
2 cos(-1+0i) = 0.5403023+0i
```

[复制代码](#)

注：项目代码链接 https://github.com/miketang84/jikeshijian/tree/master/30-ffi/rust_call_c

bindgen

Rust 官方出了一个 [bindgen](#) 项目，可以帮助我们快速从 C 库的头文件生成 Rust FFI 绑定层代码。之所以能这样做，是因为我们发现，C 的接口转换成 Rust 的接口形式是非常固定的，转换的过程需要写大量的样板代码，所以就可以用 bindgen 这种工具自动转换。

比如某个 C 头文件里定义了这样的类型和函数：

[复制代码](#)

```
1 typedef struct Doggo {
2     int many;
3     char wow;
4 } Doggo;
5
6 void eleven_out_of_ten_majestic_af(Doggo* pupper);
```

使用 bindgen 自动生成 FFI 绑定后，可以生成类似这样的代码：

[复制代码](#)

```
1 /* automatically generated by rust-bindgen 0.99.9 */
2
3 #[repr(C)]
4 pub struct Doggo {
5     pub many: ::std::os::raw::c_int,
6     pub wow: ::std::os::raw::c_char,
7 }
8
9 extern "C" {
10     pub fn eleven_out_of_ten_majestic_af(pupper: *mut Doggo);
11 }
```

你可能发现了，Rust 里有一批类型，以 `c_` 前缀开头。比如 C 语言的 `int` 类型，对应 `std` 中的类型你可以通过我给出的 [链接](#) 找到。

这些就是 Rust 里定义的与 C 完全相同的类型。我们可以看一下 [c_char](#) 的定义。

[复制代码](#)

```
1 pub type c_char = i8;
```

竟然直接是一个 `i8`。因为 C 语言里的 `char` 就是一个普通字节，和 Rust 里的 `char` 完全不同。这是在 Rust 中完全兼容 C 必要的一步，就是把 C 中的类型全部映射过来，并加上 `c_` 前

缀。这样单独搞一批类型就是因为有的类型没办法直接映射到 Rust 的 char 上来，比如刚刚说的这个 c_char。

典型的，还有 C 语言中的 void 类型，它在 Rust 里没有对应物，所以就映射成了 `c_void`，在 Rust 中定义成了一个 enum。

[复制代码](#)

```
1 #[repr(u8)]
2 pub enum c_void {
3     // some variants omitted
4 }
```

下面我们继续看，反过来，在 C 中如何调用 Rust 代码。

C 调用 Rust

在 C 中调用 Rust 代码的基本思路是，Rust 代码编译成 `.so` 或 `.dll` 动态链接库，在 C 语言里面编译的时候，连接就可以了。

在 lib.rs 文件中，写这样一个函数：

[复制代码](#)

```
1 #[no_mangle]
2 pub extern "C" fn hello_from_rust() {
3     println!("Hello from Rust!");
4 }
```

注意属性宏 `#[no_mangle]`，在 Rust 中，no_mangle 用于告诉 Rust 编译器不要修改函数的名称，这种修改叫做 Mangling，它是编译器在解析名称时，修改我们定义的函数名称，增加一些用于其编译过程的额外信息。但在和其他语言交互时，如果函数名称被编译器修改，程序开发者就没办法知道修改后的函数名称了，其他语言也无法按原名称调用。因此，`#[no_mangle]` 在这种场景下就非常有用。

Cargo.toml 要加个配置，表示编译的这个库是 C ABI 格式的动态链接库。

 复制代码

```
1 [lib]
2 crate-type = ["cdylib"]
```

然后用 `cargo build` 编译输出。你可以在 `target/debug` 目录下面看到你的 `.so` 库文件，比如下面这个样子：

 复制代码

```
1 $ ls target/debug/
2 build  deps  examples  incremental  libc_call_rust.d  libc_call_rust.so
```

这里这个 `libc_call_rust.so` 就是我们的动态链接库，其中 `c_call_rust` 是我们这个 crate 的名字。

下面是 C 的部分，代码如下：

 复制代码

```
1 extern void hello_from_rust();
2
3 int main(void) {
4     hello_from_rust();
5     return 0;
6 }
```

存成 `call_rust.c` 文件。用下面这种形式编译：

 复制代码

```
1 gcc call_rust.c -o call_rust -lc_call_rust -L./target/debug
```

运行：

```
1 LD_LIBRARY_PATH=./target/debug ./call_rust
```

你可以看到输出内容 `Hello from Rust!`。

注：完整可测试代码 [🔗 https://github.com/miketang84/jikeshijian/tree/master/30-ffi/c_call_rust](https://github.com/miketang84/jikeshijian/tree/master/30-ffi/c_call_rust)

当然，这只是最简单的形式，万里长征我们刚踏出第一步，还有各种复杂的 FFI 场景等着你去探索。

Cbindgen

在实际工作中，当你需要以 Rust 仓库为主，为其他语言导出 C 接口的时候，一般还需要生成 C 的头文件，这时，你应该会对 [🔗 cbindgen](#) 项目很感兴趣，它也算半官方的（在 Mozilla 名下），它的作用与前面的 bindgen 刚好相反，是从 Rust 代码中生成 C 可以调用的代码签名。

Rust 与 C 的交互我们先讨论到这里，下面我们要研究一下如何用 Rust 给 Python 写扩展。

使用 Rust 给 Python 写扩展

标准的 Python 扩展是用 C/C++ 写的，Python 官方有 [🔗 教程](#)，感兴趣的话你可以研究一下。我们这节课聚焦于如何用 Rust 给 Python 写扩展。

其实，只要你将 Rust 的代码扩展到 C 可以调用，那么再进一步，按 Python 官方教程那样，继续将 C 代码封装成 Python 扩展就可以了，下面我们讲的基本思路也是这样。

在实际实现的时候，我们会写非常多的样板代码，写起来比较痛苦。但是 Rust 社区有非常优秀的项目：PyO3，它可以帮助我们减轻这种烦恼，自动帮我们处理好中间样板封装过程。

PyO3

PyO3 是 Rust 社区里非常火的与 Python 绑定的框架，它不仅可以实现用 Rust 给 Python 写扩展，还能在 Rust 的二进制程序中直接执行 Python 代码。所以实际 PyO3 是一个双向绑定库。

PyO3 封装了底层 FFI 绑定的各种细节。这些细节其实不是那么简单，你可以想一下，如何将 Python 的 Class 正确映射到 Rust 中对应的类型，如何正确处理模块、函数、参数、返回值类型，这里面有各种细节，想想就头痛。

而 PyO3 把这一切都封装在了 Rust 属性宏里面。我们来看一个示例。

[复制代码](#)

```
1 use num_bigint::BigUint;
2 use num_traits::{One, Zero};
3 use pyo3::prelude::*;
4
5 // Calculate large fibonacci numbers.
6 fn fib(n: usize) -> BigUint {
7     let mut f0: BigUint = Zero::zero();
8     let mut f1: BigUint = One::one();
9     for _ in 0..n {
10         let f2 = f0 + &f1;
11         f0 = f1;
12         f1 = f2;
13     }
14     f0
15 }
16
17 #[pyfunction]
18 fn calc_fib(n: usize) -> PyResult<()> {
19     let _ = fib(n);
20     Ok(())
21 }
22
23 #[pymodule]
24 fn rust_fib(_py: Python<'_>, m: &PyModule) -> PyResult<()> {
25     m.add_function(wrap_pyfunction!(calc_fib, m)?);
26     Ok(())
27 }
```

这个示例的作用是计算斐波那契数列的第 N 项。当这个 N 值比较大的时候，斐波那契数是一个非常大的数，一个 u64 装不下，因此我们要使用大数类型。在 Rust 中，num-bigint 是一个常用的大数类型实现库。而在 Python 中，int 默认就支持大数。

代码中，`fib()` 函数是真正计算斐波那契数的函数，然后在 `calc_fib()` 中封装给了 Python 使用，请注意这个函数头上的 `#[pyfunction]` 标注，它表明这个函数会导出给 Python 使用，是一个函数。

而下面的 `rust_fib()` 函数，则是标准的样板代码，它表示创建一个 Python 的模块，这个模块的名字叫做 `rust_fib`，这个名字会在 Python 代码中以 `import rust_fib` 的形式导入。`#[pymodule]` 就起这个自动转化的作用。这个函数中的 `m` 参数就表示模块实例，`m.add_function()` 将 `calc_fib()` 添加到这个模块中。所以我们这个 crate 库会被编译成一个共识库，并被导入为 Python 的一个模块。

你可以看一下 Cargo.toml 的新增配置。

 复制代码

```
1 [lib]
2 name = "rust_fib"
3 crate-type = ["cdylib"]
```

在 Python 中这样调用：

 复制代码

```
1 import rust_fib
2 rust_fib.calc_fib(2000000)
```

注：完整的可运行代码 <https://github.com/miketang84/jikeshijian/blob/master/30-ffi/bigint-pyo3>

这样 PyO3 项目的一个关键是，需要使用 maturin 这种构建工具。你可以下载代码后，按下面的脚本准备好虚拟环境，并在这个虚拟环境里安装好 maturin。

 复制代码

```
1 $ cd bigint-pyo3
2 $ python -m venv .env
3 $ source .env/bin/activate
4 $ pip install maturin
```

在项目目录下运行：

 复制代码

```
1 maturin develop -r
```

这个指令会编译 Rust 代码为 release 模式，并把编译后的文件安装到 Python module 库的目录组织里去。

然后运行：

 复制代码

```
1 $ time python fib_rust.py
2
3 real    0m12.452s
4 user    0m12.431s
5 sys     0m0.020s
```

我们对比一下纯 Python 版本运行的时间。

 复制代码

```
1 $ time python fib.py
2
3 real    0m21.730s
4 user    0m21.490s
5 sys     0m0.240s
```

这是计算第 200 万项斐波那契数，可以看到用 Rust 实现的版本快将近一倍。

PyO3 给我们提供的编程界面非常清爽，只要你跟着操作一遍，很快就能用 Rust 给 Python 写扩展了。PyO3 的 [文档](#) 也写得非常好，上面有更全面的功能介绍，一定要读一读。

性能优化示例：文本单词统计

Python 的 GIL (Global Interpreter Lock) 是一个被人诟病的特性，它限制很多 Python 代码只能以单线程的形式来跑。因此也就大大限制了 Python 的性能。而 Rust 原生支持系统级多线程 (thread) 以及轻量级线程 (tokio task 等)，能够非常方便地充分压榨所有 CPU 核。既然我们可以使用 Rust 给 Python 写扩展，那一个点子就顺理成章冒出来了——可以使用 Rust 给 Python 写扩展实现并行计算。

下面我们就以官方的示例 word_count 来说明如何使用。这个程序要解决这样一个问题：统计出一个文本文件里，某一个单词出现的次数。这个文本文件是按换行符分隔成一行一行的，行与行之间互不干涉。因此可以采用按行分割的形式来并行化处理。

Rust 中有一个著名的并行化库 Rayon，可以将串行迭代器转换成一个并行化版本的迭代器，从而实现程序的并行化。下面我们来看代码：

[复制代码](#)

```
1 use pyo3::prelude::*;
2 use rayon::prelude::*;
3
4 /// 并行化版本的搜索功能实现
5 #[pyfunction]
6 fn search(contents: &str, needle: &str) -> usize {
7     contents
8         .par_lines()
9         .map(|line| count_line(line, needle))
10        .sum()
11 }
12
13 /// 将行按空格分割，统计目标单词数目
14 fn count_line(line: &str, needle: &str) -> usize {
15     let mut total = 0;
16     for word in line.split(' ') {
17         if word == needle {
18             total += 1;
19         }
20     }
21     total
22 }
```

```

22 }
23
24 // 导出到Python module
25 #[pymodule]
26 fn word_count(_py: Python<'_>, m: &PyModule) -> PyResult<()> {
27     m.add_function(wrap_pyfunction!(search, m)?);
28
29     Ok(())
30 }

```

代码中，`search()` 函数中 `contents.par_lines()` 这一行就是使用的 Rayon 里的 `par_lines()` 迭代器，它是 `lines()` 迭代器的并行化版本。上面的 `search` 实现对应下面的 Python 版本。

[复制代码](#)

```

1 def search_py(contents: str, needle: str) -> int:
2     total = 0
3     for line in contents.splitlines():
4         for word in line.split(" "):
5             if word == needle:
6                 total += 1
7     return total

```

你可以看一下 [🔗项目代码](#)，这个项目使用 `pytest-benchmark` 来做性能评测。要运行性能评测，你需要安装 `nox`。

[复制代码](#)

```

1 // 先安装项目依赖
2 pip install .
3 // 再安装 nox 工具
4 pip install nox

```

然后在项目根目录下运行：

```
1 nox -s bench
```

稍等片刻，我们会得到如下输出：

```
1 $ nox -s bench
2 nox > Running session bench
3 nox > Creating virtual environment (virtualenv) using python3 in .nox/bench
4 nox > python -m pip install '.[dev]'
5 nox > pytest --benchmark-enable
6 ===== test session starts =
7 platform linux -- Python 3.10.12, pytest-7.4.3, pluggy-1.3.0
8 benchmark: 4.0.0 (defaults: timer=time.perf_counter disable_gc=False min_rounds=5
9 rootdir: /home/mike/works/pyo3works/word-count
10 configfile: pyproject.toml
11 plugins: benchmark-4.0.0
12 collected 2 items
13
14 tests/test_word_count.py ..
15
16
17
18 -----
19 Name (time in ms)                Min                Max                Me
20 -----
21 test_word_count_rust_parallel    19.6740 (1.0)       50.2725 (1.0)       27.90
22 test_word_count_python_sequential 78.4158 (3.99)      91.5790 (1.82)      84.48
23 -----
24
25 Legend:
26   Outliers: 1 Standard Deviation from Mean; 1.5 IQR (InterQuartile Range) from 1s
27   OPS: Operations Per Second, computed as 1 / Mean
28 ===== 2 passed in 2.78s ==
29 nox > Session bench was successful.
```

可以看到，Rust 并行化版本比 Python 顺序化版本的平均性能提升了 3 倍左右（并行化版本消耗时间为顺序化版本的 1/3 左右），实际上被处理的文件越大，这个性能提升就越明显。

业界知名 PyO3 绑定项目介绍

目前，Rust 生态和 Python 之间已经有一些知名的项目使用 PyO3 实现绑定了，我选了几个比较不错的放在下面，你可以点击链接深入了解一下。

OpenDAL 的 Python 绑定：🔗 <https://github.com/apache/incubator-opendal/tree/main/bindings/python>

Polars 的 Python 绑定：🔗 <https://github.com/pola-rs/polars/tree/main/py-polars>

Delta Lake 的 Python 绑定：🔗 <https://github.com/delta-io/delta-rs/tree/main/python>

Arrow-Datafusion 的 Python 绑定：🔗 <https://github.com/apache/arrow-datafusion-python>

tokenizers 的 Python 绑定：

🔗 <https://github.com/huggingface/tokenizers/tree/main/bindings/python>

小结

这节课我们通过 4 个示例展示了如何在 Rust 中调用 C 函数，如何在 C 函数中调用 Rust 函数，以及在 Python 中调用 Rust 实现的模块和函数。并且通过对比 Rust 的实现版本与 Python 原生的实现版本，我们发现使用 Rust 写 Python 扩展性能更好。

Unsafe Rust 编程大量出现在底层性能的优化和与其他语言交互的场景中，体现了 Rust 这门语言一个奇特的地方：Rust 语言本身对安全性和性能的追求，不但让 Rust 本身脱颖而出，而且还反哺了 C 语言的生态，用 Rust 重新实现一部分模块，同时还能无缝地接入之前的 C 系统中，提升了之前 C 系统的安全性。并且这是一种渐进的做法，非常有利于历史遗留系统的迭代更新。

此外，Unsafe Rust 编程还可以赋能其他语言社区，将 Rust 的澎湃动力和安全扩展到了 Python、JavaScript 等其他语言。它们之前的扩展使用 C/C++ 编写，同样会存在安全性和稳定性的问题，而 Rust 解决了这个问题。

不过涉及到 Unsafe Rust 和 FFI 的场景，想要封装一个真正好用的库或扩展，还有大量的细节知识待你探索。不管怎样，我们已经踏上了安全提升和性能提升之路，有 Rust 为你输出动

力，保驾护航，你便可以放开手脚，大胆去干。

思考题

请你聊一聊，C 语言中的 char 与 Rust 中的 char 的区别在哪里？C 语言的字符串与 Rust 中的 String 区别在哪里？如何将 C 语言的字符串类型映射到 Rust 的类型中来？欢迎你把自己的思考分享到评论区，也欢迎你把这节课的内容分享给其他朋友，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (5)

最新 精选



Noah●^●

2024-01-04 来自浙江

完结!

作者回复: 真棒👍



刘丹

2024-01-01 来自广东

请问在 calc_fib 函数里，为什么 fib 的计算结果被丢弃，而返回 Ok(()) 呢？

作者回复: 只是一个示例，这里只是为了比较一下计算性能。你可以尝试改进一下这个示例。👍

共 2 条评论 >



刘丹

2024-01-01 来自广东

请问老师，pyproject.toml 这个文件是哪个工具需要的？我按本课教程操作，没有创建这个文件也能正常运行。

作者回复: 这个工程文件是 maturin 创建的 . 没有创建这个文件也能正常运行, 我还没试过. 可能maturin能识别默认配置吧.

共 3 条评论 >



Geek_72807e

2024-01-01 来自山西

基本跟完了老师的课程，还有哪些进阶内容或参考资料老师推荐一下？！

作者回复: 陈天老师的课,教你完整地做一个kv db. 挺不错的. 陈老师的课可以看作是本课程的进阶课程.

共 2 条评论 >



一带一路

2024-01-01 来自四川

Rust 与 Python 绑定未来可期！

作者回复: 未来可期！

